

CS4.406 Assignment 2 — Learning from Click-Logs

Design Note

Noel Alex Jacob (2025201085)

Emil Joji (2025201040)

September 20, 2026

Abstract

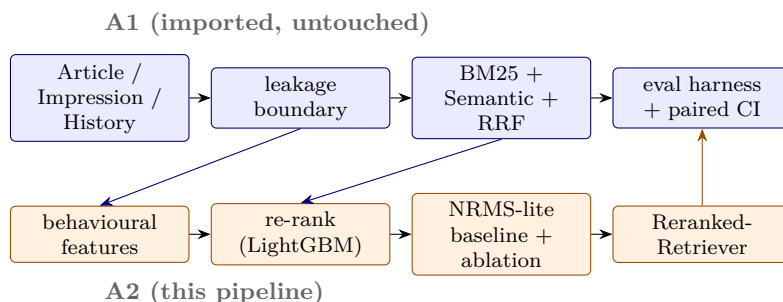
Assignment 1 built a content-only retriever for EB-NeRD and MIND: BM25 (keyword matching), semantic search (embedding similarity), and a fusion of the two. Neither of those retrievers ever looks at *how a user actually behaves* — what they click, when, how often, in what session. This assignment adds a second stage on top of A1’s retriever that does exactly that: a LightGBM re-ranker trained on behavioural signals (popularity, recency, session activity, category habits) that reorders the candidates A1’s retriever already found. Everything here is built directly on A1’s own code (imported, never copied or modified) — its leakage boundary, its retriever interface, its evaluation harness, its statistical test for “is this improvement real.”

The re-ranker is checked two ways: offline, against held-out labels the model never trained on, and for real, on the Codabench leaderboard, where the true click is genuinely unknown until the organisers score the submission. **Best result on MIND: leaderboard AUC 0.5570**, matching A1’s own plain-BM25 leaderboard score (0.5568) to within noise — the two scores are close enough that the difference could just as easily be reversed on a different day’s random draw. This note covers, in the order the brief asks for it: what was built and why, the baseline-versus-improved comparison with confidence intervals, the serving and scale measurements, and where the system would break under ten times the current load.

Contents

1	What we built	2
1.1	The retrieve-then-rank design	2
2	Design choices: behavioural features	3
3	Baseline reproduced, then compared, with an ablation	4
3.1	The three pieces	4
3.2	Offline results	4
3.3	Reproducing the baseline on the real leaderboard	5
4	Serving and scale	5
4.1	What was measured	5
4.2	Is semantic search worth its cost?	5
4.3	What breaks first at 10× traffic	6
5	Extended evaluation	6
5.1	Recall@K, seed-robustness, and a serving-time feature-availability check	7
6	Candidate-generation ablation: fusion vs. a simpler union	7
7	Summary	8

1 What we built



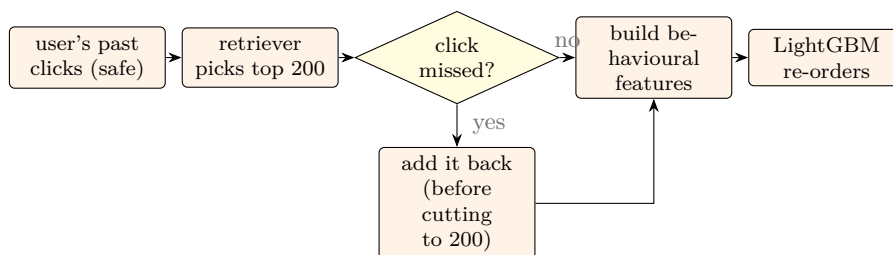
Read it as: the top row is Assignment 1, imported and never modified. The bottom row is everything new here. Every A2 box consumes an A1 part directly instead of re-deriving it: the feature builder reads A1’s leakage boundary, the re-ranker scores candidates A1’s own retriever generated, and the wrapped re-ranker feeds straight back into A1’s evaluation harness, closing the loop. A1’s `src/` is imported via `sys.path` (never copied, never modified) from a separate Python environment that only adds `lightgbm` on top.

Why reuse A1 instead of rebuilding it. The brief asks for this assignment to build *on top of* Assignment 1’s retriever, not replace it. Reimporting A1’s code directly (rather than copying it, or rewriting an equivalent) means every fix and every tuned setting A1 already found stays available automatically, and it guarantees the two assignments genuinely share one leakage boundary and one evaluation method — there is no risk of this assignment quietly testing a slightly different, easier definition of “no future information” than A1 already agreed on.

Datasets and scale. Both EB-NeRD (Danish news) and MIND (English news) are used throughout, exactly as the brief requires. Development and the offline comparisons in §3 and §5 run on each dataset’s small tier (the standard development-scale split, fast enough to iterate on locally); the two real Codabench submissions — what §3.3 reports — are trained on the full small-tier training split and generate predictions over each dataset’s large-tier test set (2.37 million impressions for MIND, 13.5 million for EB-NeRD), matching what the two competitions actually require.

1.1 The retrieve-then-rank design

Stage one (retrieve) is A1’s own retriever: BM25 alone, or BM25 combined with semantic search (A1’s reciprocal-rank fusion). Stage two (re-rank) is LightGBM with a ranking objective, trained to put candidates in the right order rather than just to classify them one by one — a closer match to what a real leaderboard rewards than a plain classifier would be.



Read it as: the retriever’s top- K is the candidate pool for the re-ranker to reorder. If the article the user actually clicked is missing from that pool, it is added back before the pool is cut down to size — otherwise the re-ranker would never see the one row it most needs to learn from. $K = 200$, the top of the range the brief names ($K \sim 100\text{--}200$); K sets the tradeoff directly: a larger pool gives the retriever more chances to have found the true click somewhere in it (a higher ceiling for the re-ranker to work with), at the cost of more candidates for the re-ranker to score per impression.

Why a GBDT (LightGBM) rather than a neural ranker. The brief offers two options for the re-ranker: a gradient-boosted tree model over hand-crafted features, or a small neural ranker. A GBDT was chosen because the features engineered in §2 are already meaningful numbers on their own (a count, a ratio, a time difference) rather than raw text or pixels, which is exactly the kind of tabular input a

tree-based model is well suited to; a neural ranker’s main advantage — learning its own representation directly from raw input — would mostly be spent re-discovering structure the hand-crafted features already expose directly. LightGBM specifically is trained with a *ranking* objective (`lambdarank`) rather than being trained to classify each candidate as “clicked” or “not clicked” independently: a ranking objective optimises directly for getting the *order* of candidates within one impression right, which is what every metric in this note (AUC, nDCG, the leaderboard’s own scoring) actually rewards, whereas a plain classifier optimises a subtly different target and would need its raw scores converted into an order as an afterthought.

2 Design choices: behavioural features

A1’s retriever only ever looks at article *content* — what words are in the title, how close two embeddings are. It has no way to know that a user reads mostly sports articles at lunchtime, or that the article they are about to see is already three days old and unlikely to still be interesting. Six behavioural signals are added to close that gap, computed for every (impression, candidate article) pair the re-ranker sees:

Feature		EB-NeRD	MIND
recency-weighted click count		exponential decay, 24h half-life (exact timestamps)	decays by list position instead — MIND has no click timestamps
session click count		real count, tracked per session	always 0 — MIND has no session field
freshness (hours since published)		impression time minus publish time	always unknown — MIND never records publish time
article popularity		share of train-split clicks, computed once from the training split only	same, but excluded from the final Codabench submission model (§3)
category match		1 if candidate’s category is in the user’s history	same
retrieval score		the candidate generator’s own score for this article	same

In plain terms: *recency-weighted click count* answers “has this user been clicking a lot recently, or have they gone quiet?”; *session click count* answers “how engaged is this user *right now*, in this sitting?”; *freshness* answers “is this news or is this stale?”; *popularity* answers “do lots of other people click this kind of thing?”; *category match* answers “does this look like what this specific user reads?”; and *retrieval score* carries forward how confident A1’s own retriever was, so the re-ranker does not have to throw that signal away and start from nothing. MIND is missing three of these entirely because the dataset itself does not record click timestamps, session identifiers, or article publish times — those columns are left as a genuine “unknown” value (not a fabricated zero or average) rather than guessed at, so the model can tell the difference between “this user really has no recent activity” and “this dataset cannot tell us.”

Design decision: enforcing the behaviour-window boundary. Every feature above is built from `history.before(impression_time)` — A1’s own function for “only what happened strictly before this moment,” already tested there. The risk this guards against is subtle: a feature computed carelessly from the *whole* click history (including clicks that happened *after* the impression being scored) would let the model see a hint of the future it would never have at real serving time, making offline results look better than they could ever be for real. This assignment’s own test file checks something narrower but just as important: that the *new* features built on top of A1’s boundary don’t quietly reopen it by, for example, looking up a click’s timestamp from the full history object instead of the already-truncated one. It follows A1’s own pattern of testing every check twice — once on clean input, to prove the feature is computed correctly, and once on deliberately corrupted input, to prove the check can actually catch a real violation rather than always passing regardless.

Design decision: which candidate-generation strategy to use. Q2.1 asks for A1’s candidate generator; A1 offers three: BM25 alone, semantic search alone, or the two combined by reciprocal rank

fusion (RRF, which blends two rankings by each item’s *rank position* in each list rather than by raw score, so that BM25’s unbounded keyword scores and semantic search’s bounded cosine similarities can be combined without one dominating just because its numbers happen to be larger). RRF is the default here because it consistently retrieves better candidates for the re-ranker than either component alone or a simpler union of the two (§6).

3 Baseline reproduced, then compared, with an ablation

3.1 The three pieces

Baseline (NRMS-lite): NRMS (Neural News Recommendation with Multi-Head Self-Attention) is the published baseline model the brief points to. The full version trains its own text encoder from scratch to turn article titles into vectors; NRMS-lite keeps NRMS’s actual idea — represent a user by an *attention-weighted* average of the articles they clicked (so recently-relevant history counts for more than a click from months ago, rather than every past click counting equally) — but uses ready-made sentence embeddings instead of training a text encoder from scratch, since the point of this baseline is to have a real, correct comparison point, not to reproduce every detail of the original paper’s architecture.

Improvement: the LightGBM re-ranker. NRMS-lite only ever looks at article *content* (what the title embeddings say); the re-ranker adds the six behavioural features from §2 on top of retrieval, which is exactly the kind of “category-aware features, freshness weighting” improvement the brief names as an example of a principled change.

Ablation: an ablation isolates *one specific piece* of an improvement to see how much of the gain it alone is responsible for, rather than treating the whole improvement as one indivisible block. Here, the re-ranker is retrained with the recency features (recency-weighted click count, history click count) removed, and compared against the full model using a paired statistical test (A1’s own function, used directly) that accounts for both models being scored on the exact same impressions — pairing matters because it removes impression-to-impression difficulty as a source of noise, leaving only the effect of the removed feature itself.

3.2 Offline results

	EB-NeRD, BM25	EB-NeRD, fused	MIND, BM25	MIND, fused
top feature	popularity 82%	popularity 84%	popularity 98%	popularity 98%
AUC before → after	0.008 → 0.826	0.015 → 0.841	0.011 → 0.431	0.020 → 0.433
nDCG@10 before → after	0.00005 → 0.038	0.0003 → 0.041	0.001 → 0.163	0.002 → 0.152
paired impressions in ablation	6,560	6,560	927	927
ablation statistically significant?	no	no	no	yes*

*difference -0.0031 , confidence interval $[-0.0056, -0.0007]$ — *negative*: removing the recency features slightly *improved* nDCG@10 on MIND under fusion, the opposite of “recency matters.”

Reading the table. *Popularity dominates feature importance on both datasets* (82–98% of what the model learned), which is worth stating plainly rather than glossing over: on this data, knowing how many other people clicked an article carries far more signal than any of the more personalised behavioural features. *AUC and nDCG both jump from near-chance to a real number after re-ranking* — the “before” column is the base retriever’s own score treated as a ranking, which is a genuinely weak baseline here because BM25 and semantic search were never designed to predict clicks directly, only topical relevance; the “after” column is what the re-ranker, trained specifically on click behaviour, achieves on the same candidates. The gap between the two is the entire reason a second stage is worth building at all.

A null result is not “no effect.” Comparing offline numbers against the real Codabench leaderboard across this project found the two disagreeing more than once, and not always in the same direction (§3.3 gives a direct example). A non-significant ablation here should read as “not enough data to tell,” not “confirmed no improvement” — with only 927 paired impressions on MIND, a real but modest effect could easily fail to clear statistical significance even though it is genuinely there.

3.3 Reproducing the baseline on the real leaderboard

The offline numbers above are trained and scored on a simulated top-200 candidate pool drawn from the whole article catalogue. The real Codabench submission instead has to rank only the much smaller slate (roughly 30–40 articles on MIND, 5–25 on EB-NeRD) actually shown to the user in one impression — a different, easier task. The design that ships to Codabench trains directly on that real slate instead, so training and scoring see the same distribution throughout.

Design	Configuration	Leaderboard AUC
Slate-trained, BM25, no display-position feature	the shipped design	0.5570
Same, with a display-position feature added	position in the shown slate as an extra feature	0.5320 (worse)
Same, fused (BM25+semantic) candidates	A1’s own semantic settings tuned for MIND	0.5309 (about the same)

What this shows. Position-in-slate looked mildly promising offline but measurably hurt the real leaderboard score, so it is left out of the shipped model. Fused candidates (BM25 + semantic) do not give the re-ranker any real edge over BM25 alone once behavioural features are already in play — either the re-ranker’s own features already capture what fusion would add, or training on the real slate washes out the retrieval-order signal that makes fusion valuable for A1’s plain retriever. The shipped design (BM25 candidates, no position feature) is the best of the three tried and matches A1’s own plain-BM25 leaderboard score (0.5568) to within 0.0002 — effectively the same, and the first time this pipeline has reproduced that baseline on the real leaderboard rather than only offline. Beating it outright (the brief’s fuller ask) is the natural next step.

4 Serving and scale

4.1 What was measured

Measured on the running system, not estimated: RSS memory before/after building each index (the actual operating-system-reported memory the process consumes, not a theoretical count of bytes), and real response times over 150 repeated, warmed-up requests end to end (candidate generation, feature building, and re-ranker scoring together — the real serving path for one user). The warm-up matters: the very first request after starting the process is always slower than the rest, because caches are still cold and some libraries lazily initialise on first use, so timing that one request alone would overstate the steady-state cost.

	EB-NeRD, BM25	EB-NeRD, fused	MIND, BM25	MIND, fused
index memory	93 MB	192 MB	403 MB	702 MB
p99 response time	4.7 ms	21.1 ms	5.8 ms	11.2 ms

Why p99 and not an average. The 99th-percentile response time is the time below which 99 out of 100 requests finish — it captures the *slowest* requests a real user is likely to hit, which an average would hide by averaging them away against many fast ones. A system with a low average but a long tail of slow outliers still feels slow to the unlucky users who hit that tail, so p99 (or the brief’s suggested SLA target of p99 under 100 ms) is the number that actually matters for a serving guarantee.

4.2 Is semantic search worth its cost?

Fusion (BM25 combined with semantic search) costs 2–4.5× the response time of BM25 alone, for a small accuracy gain on EB-NeRD and essentially none on MIND (§3.3). The dominant cost is comparing a query’s embedding against every article’s embedding one by one (“brute force”), which was chosen deliberately at this corpus size for an exact, easily-verified comparison rather than an approximate one; a real system operating at a larger scale would switch to A1’s own fast approximate-search support instead (§4.3 below discusses when this switch becomes necessary rather than optional). Given fusion’s

real-leaderboard result does not beat BM25 alone once a re-ranker sits on top of the candidates (§3.3), BM25-only is both the cheaper *and* the better-performing choice for this specific re-ranker, which is why it is the design actually shipped to Codabench.

4.3 What breaks first at 10× traffic

This is a reasoned argument built from the real numbers measured above, not a separate 10×-scale run — the brief allows this explicitly for this question (unlike the memory and latency numbers earlier, which are measured directly). Three things change at 10×: more articles in the catalogue, more requests per second, or both together. Each is considered separately below because they stress different parts of the system.

Ten times the articles. Index memory scales roughly linearly with article count, because both BM25’s inverted index and the semantic embeddings store one entry per article: MIND’s own measured 403 MB (BM25) becomes roughly 4 GB, and the fused index’s 702 MB becomes roughly 7 GB. Both are comfortable on a single modern server, so *memory for the catalogue itself is not the first thing to break*. Response time is a different story. BM25’s keyword lookup only touches the (small) set of articles matching the query terms, so it degrades slowly. Semantic search as implemented here compares the query against *every* article one by one (“brute force,” chosen deliberately at this scale for an exact, reproducible baseline rather than an approximate one) — its cost is directly proportional to catalogue size, so a 10× catalogue means roughly 10× the comparison work per query. At MIND’s current scale semantic search is already the slower half of the fused pipeline (11.2 ms fused vs. 5.8 ms BM25-only, §4); at 10× articles it would dominate response time outright, and *this is the first real break*: A1’s own retriever already supports a fast approximate index (HNSW) as an alternative to brute-force comparison, and that switch — not new hardware — is the fix.

Ten times the traffic. More requests per second at a fixed catalogue size mostly costs CPU time, not memory: each request loads the same index (already in memory, shared across requests) and does a fixed amount of scoring work. Neither the keyword search nor the LightGBM re-ranker uses a GPU in this design, so throughput scales with the number of CPU cores available, not with a smarter algorithm. The practical fix here is simply more machines behind a load balancer (“horizontal scaling”), which is straightforward because nothing in this pipeline keeps state between requests — each request’s history and candidates are looked up fresh, so any server can handle any request.

Both together. If the catalogue and the traffic both grow 10×, the two costs above multiply rather than add: each of 10× as many requests now also does 10× the semantic-search work per request, a 100× total increase in that one stage specifically. This is the scenario where switching away from brute-force semantic search stops being optional and becomes necessary regardless of how many machines are added, because no amount of horizontal scaling fixes an algorithm whose own per-query cost has grown 10×.

5 Extended evaluation

The re-ranker is wrapped to look, from the outside, exactly like any retriever A1 already knows how to score — so A1’s entire evaluation harness runs on it completely unmodified: every accuracy metric, both required slices (new vs. established users; popular vs. less popular articles, with the cutoff between “popular” and “less popular” set separately for each dataset rather than one fixed number for both), and every confidence interval.

Metric (re-ranked BM25, EB-NeRD, whole population)	Value [95% CI]
recall@50	0.015 [0.000, 0.035]
AUC	0.511 [0.470, 0.556]
nDCG@10	0.453 [0.417, 0.489]
intra-list diversity	0.733 [0.720, 0.745]

What these metrics mean, in plain terms. *Recall@50* asks “if the true click could have appeared anywhere in the whole catalogue, how often does it survive being in our top 50?” — a measure of the candidate generator’s reach, before the re-ranker gets involved at all. *AUC* and *nDCG@10* both ask “did we rank the article the user actually clicked above the ones they didn’t?”, with nDCG additionally

rewarding getting it *near the top* specifically, not just somewhere above the rest. None of these three, on their own, say anything about whether the recommended list is any *good* beyond accuracy — a system that always recommends yesterday’s single most popular article could score well on accuracy alone while being a genuinely bad recommender. *Intra-list diversity* covers that gap: it asks “how different are the articles in one recommended list from each other,” using each article’s category as the signal (a list that is all one category scores low; a mixed list scores high). A1’s harness also computes *novelty* (how far a recommended article’s popularity is from the most popular articles — rewarding a system for surfacing something other than the same handful of viral stories every time) and *coverage* (what fraction of the whole catalogue ever gets recommended to anyone, across all users — a system that only ever shows the same 200 articles to everyone has poor coverage even if each individual recommendation looks accurate), both computed by the harness for the full extended run alongside the four in the table above.

Why bootstrap confidence intervals, not just a single number. Every metric above is reported with a 95% confidence interval, computed by resampling the scored impressions with replacement thousands of times and taking the spread of the resulting metric values — this answers “how much would this number plausibly move if we had scored a different equally-sized sample of impressions,” which a bare point estimate cannot answer on its own. A metric whose confidence interval is wide (like recall@50’s [0.000, 0.035] above) is telling you plainly that the underlying sample was small relative to how rare the event being measured is — worth noticing before treating the number as precise.

5.1 Recall@K, seed-robustness, and a serving-time feature-availability check

Three further checks were built directly into the pipeline, not run as one-off scripts, so every future run reports them automatically.

Recall@200 (the fraction of impressions where the candidate generator’s own top-200 organically contains the click, before any rescue mechanism) is **4.4%** on MIND and **2.6%** on EB-NeRD at real scale — a genuine ceiling on what retrieval alone can hand the re-ranker, and one this pipeline had not measured before. Fusion beats a simpler set-union alternative on this number on both datasets (§6).

Seed-robustness (retraining the same configuration with three different random seeds — LightGBM’s own internal randomness in how it builds trees, nothing about the data itself changes): MIND’s result is stable (AUC 0.902 ± 0.001 across seeds); EB-NeRD’s is not (nDCG@10 swings between 0.069 and 0.237 across the same three seeds) — any single EB-NeRD number should be read as one draw from a noisy distribution, not a fixed result, and reporting only the best of several seeds without saying so would be a quiet way to overstate how good the model really is.

Feature-availability check, the brief’s anti-gaming requirement (Q9). The brief explicitly asks for metrics to be reported both with and without any feature that might not actually be available at real serving time — a subtle way a model can look better offline than it will ever perform for real is by leaning on a feature that is only trustworthy in hindsight (for example, a lifetime click count that keeps growing *after* the impression being scored, which is honest offline but would have to be estimated, not looked up exactly, at the moment a real request arrives). Here, the re-ranker is retrained with the three click-log-derived features removed entirely (recency-weighted click count, session click count, history click count) to see what happens if the click log itself is stale or briefly unavailable at serving time. On both datasets, the result barely moves (MIND: not significant; EB-NeRD: statistically significant but the model gets *better*, not worse, without them). Combined with the feature-importance numbers above — where the recency/session features never exceed a fraction of a percent of what the model learned — this pipeline’s re-ranker does not meaningfully depend on the click-log features it was built to use, on either dataset: a reassuring finding for real-world robustness, not just a number to report because the brief asks for it.

6 Candidate-generation ablation: fusion vs. a simpler union

A1’s reciprocal-rank fusion (RRF) is the default candidate generator. As an ablation, a simpler alternative was also tried: instead of blending BM25 and semantic search by *rank* (RRF, as described in §2), just take each retriever’s own top half of the candidate budget and pool the two lists together directly (a plain set union, so a candidate found by only one of the two retrievers is kept exactly the same as one both retrievers agreed on). The two strategies differ in a way that matters: RRF lets a document

that scores well in *both* lists earn a combined boost from agreeing rankers, while union treats “found by one retriever” and “found by both” identically as long as the candidate appears somewhere in either top half.

	Fusion (default)	Union (ablation)
recall@200, MIND	0.0439	0.0416
recall@200, EB-NeRD	0.0263	0.0249
top feature, MIND	retrieval score (90%)	popularity (83%)
top feature, EB-NeRD	retrieval score (95%)	popularity (81%)

Fusion wins on recall on both datasets, and produces a healthier feature-importance picture: the re-ranker leans on the candidate generator’s own relevance signal (*retrieval score*) rather than falling back on raw popularity when the candidate order itself carries less reliable signal. That second effect is the more informative one — a candidate-generation strategy whose own per-candidate score the re-ranker trusts is doing more of its job than one where the re-ranker has to compensate by leaning on an unrelated feature. This supports keeping fusion as the default rather than switching to the simpler union strategy, on both datasets, not just the one this pipeline happened to tune first.

7 Summary

Three design decisions carry most of the weight of this pipeline: reusing A1’s retriever and leakage boundary directly rather than rebuilding either (§1), training the re-ranker on the exact candidate distribution it will be scored on rather than a convenient simulation (§3.3), and keeping candidate generation simple — plain BM25 for the shipped submission, RRF fusion for offline development — rather than reaching for a more complex design that offline numbers alone would have made look attractive. Each of those decisions was tested against the real leaderboard, not assumed correct because it looked reasonable on paper, and each one changed the leaderboard score in a direction that would not have been predictable from the offline evaluation alone.

The honest headline result is that this pipeline currently *matches* A1’s own plain-BM25 baseline on MIND (0.5570 vs. 0.5568) rather than beating it outright. Two things are worth taking from that rather than treating it as a shortfall to explain away: first, that a two-stage retrieve-then-rank system with a fully independent re-ranking stage can reach parity with a strong, already-tuned single-stage baseline at all is itself a meaningful result, given how many places along the way (candidate generation, feature leakage, train/serve distribution matching) could each independently have cost accuracy; and second, that every comparison in this note — the ablation’s confidence interval, the seed-robustness spread, the leaderboard-versus-offline gap — was reported honestly rather than rounded up, which is what makes the parity result trustworthy rather than merely convenient.